🚨 **DevOps Monitoring & Troubleshooting Flow (Real Production Deep Understanding**

🚨 **1. ALERT TRIGGER (FIRST SIGNAL)**
👉 Production system lo problem start ayithe monitoring tools automatic ga alert generate chestayi.
👉 Prometheus, Grafana, CloudWatch laanti tools DevOps engineer ki notification pampistayi.
👉 Alert lo **CPU high, memory high, pod crash, service down, high error rate** laanti information untundi.
📌 **Explanation:** System itself problem ni detect chesi engineer ki early warning istundi.
👉 **Example:**
Suppose production API ki requests fail avuthunnayi anuko. Prometheus error rate increase ni detect chesi Slack lo **"API Error Rate > 10%"** ani alert pampistundi.
🔍 **Ela Check Cheyyali?**
👉 First alert eppudu start ayindo timestamp check cheyyali.
👉 Which service affected ayindo identify cheyyali.
👉 Alert firing lo unna metric exact value check cheyyali.
👉 Recent deployment or infrastructure change jariginda ani check cheyyali.
🔧 **Ela Fix Cheyyali?**
👉 Alert genuine problem ayithe immediate incident investigation start cheyyali.
👉 False alert ayithe monitoring rule or threshold correct cheyyali.
✅ **Ela Verify Cheyyali?**
👉 Alert resolve ayinda ani check cheyyali.
👉 Metric normal range ki vachinda chudali.
👉 Application users ki properly respond avtunda verify cheyyali.
🖥️ **Useful Commands**

```
# Kubernetes pods check
kubectl get pods -A

# Kubernetes nodes check
kubectl get nodes

# Current events check
kubectl get events -A --sort-by=.lastTimestamp

# Prometheus / monitoring pods check
kubectl get pods -n monitoring

# Kubernetes deployment status
kubectl get deployments -A
```

📌 **Real production idea:** Alert ante solution kaadu; **problem ekkada start ayindo investigate cheyyadaniki first signal.**

📊 **2. MONITORING DASHBOARD CHECK CHEYYADAM**
👉 Alert vachina tarvata DevOps engineer Grafana / CloudWatch dashboard open chesi complete system health check chestadu.
👉 CPU usage, memory usage, request count, latency and error rate observe chestaru.
👉 CPU 100% ayithe server heavy load lo undani ardham.
👉 Memory 90–100% ayithe application ki RAM pressure undani ardham.
👉 Latency sudden ga increase ayithe application slow ga respond avtundi.
👉 Error rate spike ayithe backend lo problem undochu.
📌 **Explanation:** Guessing kakunda metrics chusi problem direction identify chestaru.
🏠 **Real Example**
👉 Suppose API response normally **200 ms** untundi.
👉 Suddenly latency **200 ms → 5 seconds** ki increase ayindi.
👉 Same time CPU 95% ki vellindi.
👉 Error rate kuda increase avtundi.

👉 Ippudu CPU saturation problem undochu ani first direction vastundi.
🔍 **Ela Check Cheyyali?**
👉 CPU graph check cheyyali.
👉 Memory graph check cheyyali.
👉 Request count sudden ga increase ayinda chudali.
👉 Latency graph check cheyyali.
👉 Error rate graph compare cheyyali.
👉 Problem start ayina time ni identify chesi other metrics tho correlate cheyyali.
🔧 **Ela Fix Cheyyali?**
👉 Traffic spike ayithe Auto Scaling / HPA scale out cheyyali.
👉 Application CPU-intensive operation ayithe code optimize cheyyali.
👉 Single server overload ayithe additional capacity provide cheyyali.
✅ **Ela Verify Cheyyali?**
👉 CPU normal range ki vachinda check cheyyali.
👉 Latency malli normal ayinda chudali.
👉 Error rate reduce ayinda verify cheyyali.
👉 User requests successful ga complete avtunnaya check cheyyali.
🖥️ **Useful Commands**

```
# Kubernetes node resource usage
kubectl top nodes

# Pod CPU and memory usage
kubectl top pods -A

# Specific namespace pods resource usage
kubectl top pods -n <namespace>

# Deployment replica status
kubectl get deployment <deployment-name> -n <namespace>

# HPA status
kubectl get hpa -A

# Detailed HPA information
kubectl describe hpa <hpa-name> -n <namespace>
```

📌 **Real production idea:** Dashboard answer ivvadu; **problem direction identify cheyyadaniki evidence istundi.**

📜 **3. LOGS CHECK CHEYYADAM**
👉 Metrics problem undani cheptayi, kani exact reason chala sarlu logs lo dorukutundi.
👉 Anduke system logs open chestaru using **CloudWatch Logs, ELK, kubectl logs** etc.
👉 Logs lo application inside lo em jarigindo detailed information untundi.
👉 Timeout, database failure, authentication error, application crash laanti errors identify cheyyachu.
📌 **Explanation:** Logs root cause ki strong evidence provide chestayi.
🏠 **Real Example**
👉 Dashboard lo API error rate high ani kanipinchindi.
👉 Logs check chesthe:
Database connection timeout
👉 Ee error repeatedly kanipistundi.
👉 App problem laga kanipinchina actual problem database connection ayye chance undi.
🔍 **Ela Check Cheyyali?**

```
# Pod logs
kubectl logs <pod-name> -n <namespace>

# Last 100 log lines
kubectl logs <pod-name> -n <namespace> --tail=100
```

```
# Follow live logs
kubectl logs -f <pod-name> -n <namespace>

# Previous crashed container logs
kubectl logs <pod-name> -n <namespace> --previous

# Logs from all pods matching a label
kubectl logs -l app=<app-name> -n <namespace>
```

🔧 **Ela Fix Cheyyali?**

👉 Database unreachable ayithe network/security configuration check cheyyali.

👉 Connection pool full ayithe connection configuration optimize cheyyali.

👉 Database overloaded ayithe database capacity or query performance check cheyyali.

👉 Application bug ayithe code fix chesi new version deploy cheyyali.

✅ **Ela Verify Cheyyali?**

```
# Follow new logs
kubectl logs -f <pod-name> -n <namespace>

# Check pod status
kubectl get pods -n <namespace>

# Check recent events
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

📌 **Real production idea: Metrics “problem undi” ani cheptayi; logs “problem enduku vachindi” ani cheppadaniki help chestayi.**

☸️ **4. KUBERNETES POD CHECK**

👉 Application Kubernetes lo run avutunte pods status check chestaru.

👉 Pods **Running, Pending, Failed, CrashLoopBackOff** state lo unnaya chustaru.

👉 Repeated crash ayithe application instability undani ardham.

👉 Worker node health kuda verify chestaru.

📌 **Explanation:** Application problem aa, pod problem aa, node problem aa ani separate cheyyali.

🏠 **Real Example**

👉 Alert vachindi: **Payment service unavailable**.

👉 kubectl get pods check chesthe payment pod:
CrashLoopBackOff

👉 App repeatedly start avvadaniki try chestundi kani crash avtundi.

🔍 **Ela Check Cheyyali?**

```
# Check all pods
kubectl get pods -A

# Check specific namespace
kubectl get pods -n <namespace>

# Detailed pod information
kubectl describe pod <pod-name> -n <namespace>

# Check pod logs
kubectl logs <pod-name> -n <namespace>

# Check previous crash logs
kubectl logs <pod-name> -n <namespace> --previous

# Check nodes
kubectl get nodes
```

Detailed node information
kubectl describe node <node-name>

🔧 **Ela Fix Cheyyali?**

👉 Logs lo **OOMKilled** unte memory limit/request review cheyyali.

👉 ImagePullBackOff unte image name/tag/registry access check cheyyali.

👉 Configuration error unte ConfigMap/Secret/config correct cheyyali.

👉 Application bug ayithe code fix chesi new image deploy cheyyali.

✅ **Ela Verify Cheyyali?**

Pod should become Running/Ready
kubectl get pods -n <namespace>

Watch pod recovery
kubectl get pods -n <namespace> -w

Check restart count
kubectl get pods -n <namespace>

Check deployment rollout
kubectl rollout status deployment/<deployment-name> -n <namespace>

📌 **Real production idea:** Pod restart cheyyadam matrame solution kaadu. **Why pod crash ayindo identify chesi root cause fix cheyyali.**

🌐 **5. NETWORK / SERVICE CHECK**

👉 Application pods healthy ga unna requests backend ki reach avvakapothe network layer check cheyyali.

👉 Kubernetes Service, Ingress and Load Balancer configuration inspect chestaru.

👉 DNS and routing kuda check chestaru.

📌 **Explanation:** Application healthy ga unna network configuration wrong ayithe users ki service down laga kanipinchachu.

🏠 **Real Example**

👉 Payment pods anni Running lo unnayi.

👉 Kani users payment page open chesthe **502 Bad Gateway** vastundi.

👉 Pods problem kaadu.

👉 Service or Ingress routing problem undochu.

🔍 **Ela Check Cheyyali?**

Check services
kubectl get svc -A

Check service details
kubectl describe svc <service-name> -n <namespace>

Check service endpoints
kubectl get endpoints <service-name> -n <namespace>

Check EndpointSlices
kubectl get endpointslices -n <namespace>

Check ingress
kubectl get ingress -A

Detailed ingress information
kubectl describe ingress <ingress-name> -n <namespace>

Check pods and labels
kubectl get pods -n <namespace> --show-labels

🔧 **Ela Fix Cheyyali?**

👉 Wrong Service selector unte correct pod labels ki match cheyyali.

👉 Ingress routing wrong unte correct backend service configure cheyyali.
👉 Load Balancer unhealthy targets unte health check configuration investigate cheyyali.
👉 Security Group / Network ACL issue unte required traffic allow cheyyali.
✅ **Ela Verify Cheyyali?**

```
# Test service from inside cluster
kubectl run network-test --rm -it --image=curlimages/curl -- sh

# Inside test container
curl http://<service-name>.<namespace>.svc.cluster.local:<port>

# Check service endpoints
kubectl get endpoints <service-name> -n <namespace>

# Check ingress status
kubectl get ingress -n <namespace>
```

📌 **Real production idea: Pod healthy undadam alone saripodu; user request complete path lo correct ga travel avvali.**

🔍 **6. ROOT CAUSE ANALYSIS**
👉 Metrics, logs, pods, nodes and network information anni kalipi actual root cause identify chestaru.
👉 Common causes:
👉 Memory leak
👉 Sudden traffic spike
👉 Bad deployment
👉 Wrong configuration
👉 Database failure
👉 Network issue
📌 **Explanation:** Guessing kakunda evidence based decision teesukovali.
🏠 **Real Example**
👉 10:00 AM ki new deployment jarigindi.
👉 10:05 AM ki error rate suddenly increase ayindi.
👉 Logs lo new application error kanipinchindi.
👉 Old version lo same error ledu.
👉 Ippudu recent deployment root cause ayye strong evidence undi.
🔍 **Ela Check Cheyyali?**

```
# Deployment history
kubectl rollout history deployment/<deployment-name> -n <namespace>

# Current deployment status
kubectl rollout status deployment/<deployment-name> -n <namespace>

# Deployment details
kubectl describe deployment <deployment-name> -n <namespace>

# Recent Kubernetes events
kubectl get events -n <namespace> --sort-by=.lastTimestamp

# Check pod restarts
kubectl get pods -n <namespace>

# Check current image
kubectl get deployment <deployment-name> -n <namespace> \
 -o=jsonpath='{.spec.template.spec.containers[*].image}'
```

🔧 **Ela Fix Cheyyali?**
👉 Bad deployment confirm ayithe previous stable version ki rollback cheyyali.
👉 Configuration issue ayithe configuration correct cheyyali.

👉 Resource issue ayithe capacity/scaling adjust cheyyali.
👉 Code bug ayithe permanent code fix prepare cheyyali.
✅ **Ela Verify Cheyyali?**

```
# Verify rollout
kubectl rollout status deployment/<deployment-name> -n <namespace>

# Check pods
kubectl get pods -n <namespace>

# Check application logs
kubectl logs <pod-name> -n <namespace>

# Check recent errors/events
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

📌 **Real production idea:** Root cause analysis ante **"symptom ni fix cheyyadam"** kaadu; **"symptom enduku vachindo prove cheyyadam."**

🔧 **7. FIX / RECOVERY ACTION**
👉 Root cause identify ayyaka immediate recovery start cheyyali.
👉 Production incident lo first priority user impact reduce cheyyadam.
👉 Deployment issue ayithe rollback cheyyachu.
👉 Traffic spike ayithe scale out cheyyachu.
👉 Pod crash ayithe root cause fix chesi healthy pod create cheyyali.
👉 Configuration issue ayithe correct configuration deploy cheyyali.
📌 **Explanation:** System ni fast ga stable state ki bring back cheyyadam immediate goal.
🏠 **Real Example**
👉 New application version deploy chesaka payment failures increase ayyayi.
👉 Root cause new deployment ani confirm ayindi.
👉 Immediate ga previous stable version ki rollback chestaru.
👉 Traffic old version ki return avtundi.
👉 Users payment service malli use cheyyagalru.
🔍 **Ela Check Cheyyali?**

```
# Check current deployment
kubectl get deployment <deployment-name> -n <namespace>

# Check rollout history
kubectl rollout history deployment/<deployment-name> -n <namespace>

# Check running pods
kubectl get pods -n <namespace>
```

🔧 **Ela Fix Cheyyali?**

```
# Roll back deployment
kubectl rollout undo deployment/<deployment-name> -n <namespace>

# Roll back to a specific revision
kubectl rollout undo deployment/<deployment-name> \
 --to-revision=<revision-number> -n <namespace>

# Scale deployment
kubectl scale deployment/<deployment-name> \
 --replicas=<number> -n <namespace>

# Restart deployment when appropriate
kubectl rollout restart deployment/<deployment-name> -n <namespace>
```

👉 Configuration issue ayithe correct config apply chesi redeploy cheyyali.
👉 Permanent bug ayithe **code fix → test → new image → deployment** process follow cheyyali.

✅ **Ela Verify Cheyyali?**

```
# Verify rollout
kubectl rollout status deployment/<deployment-name> -n <namespace>

# Check pod health
kubectl get pods -n <namespace>

# Check application logs
kubectl logs <pod-name> -n <namespace>

# Test application endpoint
curl -I https://<application-domain>

# Check recent events
kubectl get events -n <namespace> --sort-by=.lastTimestamp
```

📌 **Real production idea: Immediate recovery + permanent fix** rendu different steps. First service ni recover cheyyali, tarvata actual permanent solution implement cheyyali.

🔁 **8. POST INCIDENT REVIEW**

👉 System recover ayyaka incident complete ga analyse chestaru.
👉 Issue enduku vachindi, entha time impact ayyindi, entha users affected ayyaru ani document chestaru.
👉 Ee analysis ni usually **Postmortem / RCA document** lo record chestaru.
👉 Same issue future lo repeat kakunda prevention steps create chestaru.
📌 **Explanation:** Incident close cheyyadam final step kaadu; future prevention kuda important.
🏠 **Real Example**
👉 Oka pod memory leak valla repeated ga crash ayindi.
👉 Temporary ga memory limit increase chesi service recover chesaru.
👉 Kani permanent solution kosam application memory leak identify cheyyali.
👉 Monitoring lo memory usage alert improve cheyyali.
👉 Deployment testing lo memory behaviour check add cheyyali.
🔍 **Ela Check Cheyyali?**

```
# Review deployment history
kubectl rollout history deployment/<deployment-name> -n <namespace>

# Review pod restart count
kubectl get pods -n <namespace>

# Review events
kubectl get events -n <namespace> --sort-by=.lastTimestamp

# Review resource usage
kubectl top pods -n <namespace>
kubectl top nodes
```

🔧 **Ela Fix Cheyyali?**

👉 Code issue ayithe permanent code fix cheyyali.
👉 Monitoring weak ayithe new alerts add cheyyali.
👉 Deployment issue ayithe CI/CD validation improve cheyyali.
👉 Capacity issue ayithe scaling rules improve cheyyali.
✅ **Ela Verify Cheyyali?**

```
# Confirm all pods are healthy
kubectl get pods -n <namespace>

# Confirm deployment is stable
kubectl rollout status deployment/<deployment-name> -n <namespace>

# Continue monitoring logs
```

kubectl logs -f <pod-name> -n <namespace>

Verify resource usage
kubectl top pods -n <namespace>

Verify HPA
kubectl get hpa -n <namespace>

📌 **Real production idea: Good DevOps engineer problem ni fix cheyyadam tho stop avvadu; same problem malli raakunda system ni improve chestadu.**

🔥 **FINAL REAL TROUBLESHOOTING FLOW**

👉 **Alert** → Problem signal vachindi
👉 **Dashboard** → Which metric abnormal ani identify cheyyali
👉 **Logs** → Exact error evidence collect cheyyali
👉 **Kubernetes** → Pod / Node health check cheyyali
👉 **Network** → Request correct destination ki reach avtunda check cheyyali
👉 **Root Cause** → Actual reason prove cheyyali
👉 **Fix / Recovery** → Immediate service recovery cheyyali
👉 **Postmortem** → Permanent fix + future prevention implement cheyyali

---

🧠 **FINAL CLEAR IDEA**

👉 DevOps troubleshooting ante **"Alert vachinappudu random ga server restart cheyyadam kaadu."**

👉 First **metrics chudali**, tarvata **logs check cheyyali**, tarvata **pods/nodes/network inspect cheyyali**, evidence tho **root cause identify cheyyali**, immediate **recovery cheyyali**, finally **permanent fix implement chesi same issue malli raakunda monitoring and system improve cheyyali.**

🔥 **Real production troubleshooting = Detect → Investigate → Identify → Recover → Verify → Prevent**